Dijkstra's algorithm

Single-source shortest paths, without a heap and with a min-heap

The single-source shortest paths problem (SSSP) asks, for a directed graph with arc costs and a source vertex s, for the shortest distance from s to every vertex and for a path that achieves it. In 1959 Dijkstra published a greedy algorithm for the case in which no arc has a negative cost [1]. This lab implements it in two versions, with a linear search for the minimum and with a min-heap, and compares their running times on large graphs. Lab 2 deals with negative costs, through the Bellman-Ford algorithm.

Contents

1 Objectives 1

2 Background 2

3 Dijkstra's algorithm 2

3.1 The version without a heap 5

3.2 The version with a min-heap 5

3.3 Which version, and when 7

4 The graph file and the generator 8

5 Lab work 8

6 Exercises 9

7 Applied questions 10

8 Review questions 10

9 Source code 10

10 References 23

1 Objectives

- Understanding the SSSP problem, the relaxation step and the shortest-path tree.
- Following Dijkstra's algorithm step by step, and the reason it needs non-negative costs.
- Implementing the two versions in C++: without a heap, in $O(n^2 + m)$, and with a min-heap, in $O((n + m) \log n)$.
- Working with graphs read from a file, saved, or generated at random.

- Measuring running times and estimating their growth with the doubling experiment.

2 Background

Let G = (V, A) be a directed graph with n = |V| vertices and m = |A| arcs, in which every arc u → v has a cost c(u, v). The cost of a path is the sum of the costs of its arcs. The distance δ(s, v) is the smallest cost of a path from s to v, or ∞ if v cannot be reached from s.

The algorithms keep, for every vertex, an estimate `d[v]` of the distance, initially 0 for the source and ∞ for every other vertex, and the predecessor `pred[v]`, the vertex before v on the best path found so far. The only operation that changes these values is the relaxation of an arc:

```
relax(u → v): if d[u] + c(u, v) < d[v], then d[v] ← d[u] + c(u, v) and pred[v] ← u
```

Relaxation never increases a distance, and `d[v]` is always the cost of a real path, so d[v] ≥ δ(s, v). At the end, the arcs pred[v] → v form the shortest-path tree rooted at s: the path to t is rebuilt by walking back through `pred[]` from t to s. This works because every part of a shortest path is itself a shortest path.

The graph is stored as adjacency lists: for every vertex, the arcs that leave it, in the order of the file. The space used is O(n + m), and going through the arcs of a vertex costs its out-degree.

3 Dijkstra's algorithm

The algorithm splits the vertices into final and unfinished ones. At every step it picks the unfinished vertex u with the smallest `d[u]`, makes it final and relaxes the arcs that leave it. A final vertex never changes again.

```
d[s] ← 0; d[v] ← ∞ for v ≠ s
while some reached vertex is unfinished:
  u ← the unfinished vertex with the smallest d[u]; u becomes final
  for every arc u → v with v unfinished: relax(u → v)
```

If no arc has a negative cost, the chosen vertex already has its final distance, d[u] = δ(s, u). Suppose there were a shorter path P to u, and let y be the first unfinished vertex on P. The predecessor of y on P is final, so the arc into y has been relaxed, and d[y] ≤ the cost of P up to y ≤ the cost of P < d[u], because the arcs after y do not lower the cost. Then y would have been chosen before u, a contradiction.

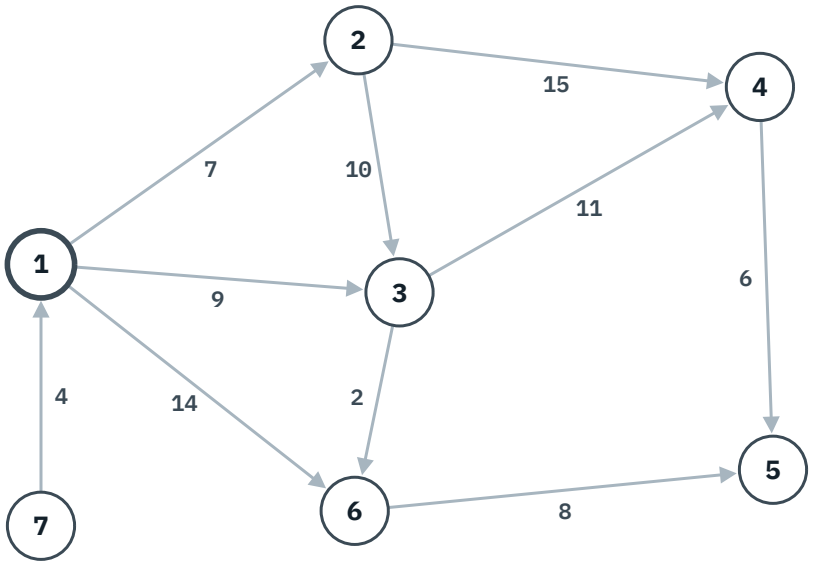
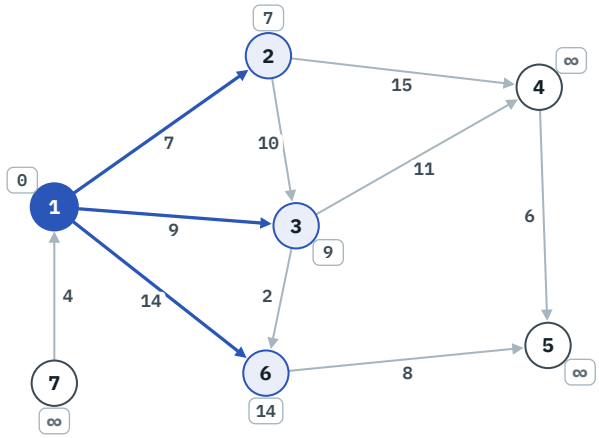
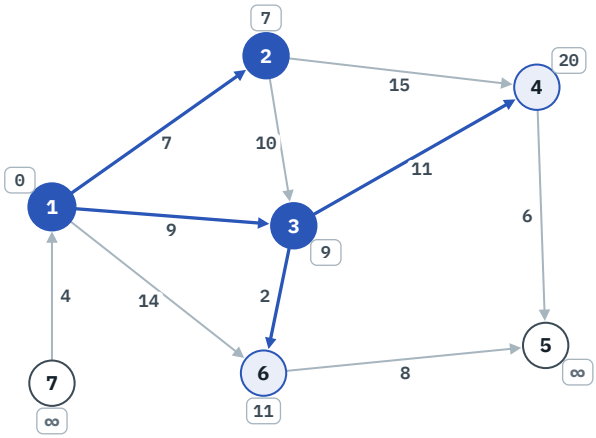


Figure 1. The small graph of the lab, from the file `graf_mic.txt`: 7 vertices and 10 arcs, source 1. Vertex 7 cannot be reached from 1: the arc 7 → 1 leaves it, but no arc enters it.

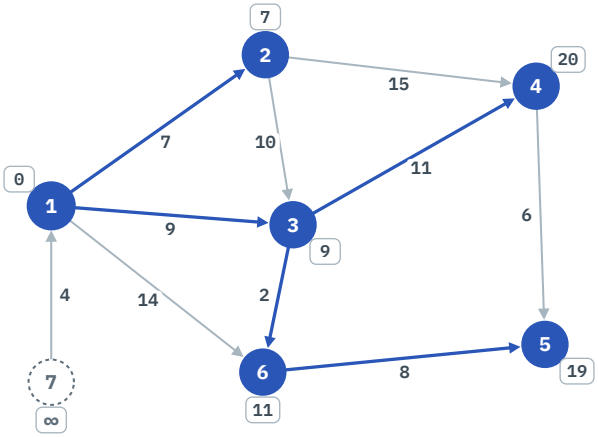
In Figure 1, step 1 extracts the source and relaxes the arcs to 2, 3 and 6: d[2] = 7, d[3] = 9, d[6] = 14. Step 2 extracts 2: the arc 2 → 3 does not help, since 7 + 10 = 17 ≥ 9, and the arc 2 → 4 gives d[4] = 22. Step 3 extracts 3, and two distances drop, d[4] = 9 + 11 = 20 and d[6] = 9 + 2 = 11. Next comes 6, which reaches 5 with d[5] = 19, then 5 and 4. The arc 4 → 5 leads to a vertex that is already final and is skipped. Vertex 7 keeps d[7] = ∞ (Figure 2, Table 1).



(a) after step 1: vertex 1 is final



(b) after step 3: d[4] and d[6] dropped



(c) at the end: the blue arcs form the shortest-path tree

Figure 2. Dijkstra's algorithm on the graph of Figure 1. Final vertices are filled, reached unfinished ones have a light blue background, and the tag next to each vertex is `d[v]`.

Table 1. The distances after every step, on the graph of Figure 1. Values in bold changed at that step, and the star marks the extracted vertex. The last column shows the min-heap of section 3.2, as a list of vertex(d) pairs, from the root to the last position.

STEP	EXTRACTED	d[1]	d[2]	d[3]	d[4]	d[5]	d[6]	d[7]	HEAP AFTER THE STEP
0	initially	0	∞	∞	∞	∞	∞	∞	1(0)
1	1	0⋆	**7**	**9**	∞	∞	**14**	∞	2(7) 3(9) 6(14)
2	2	0	7⋆	9	**22**	∞	14	∞	3(9) 6(14) 4(22)
3	3	0	7	9⋆	**20**	∞	**11**	∞	6(11) 4(20)
4	6	0	7	9	20	**19**	11⋆	∞	5(19) 4(20)
5	5	0	7	9	20	19⋆	11	∞	4(20)
6	4	0	7	9	20⋆	19	11	∞	—

On equal distances, both versions of the program pick the vertex with the smaller index. They therefore extract the vertices in the same order and give the same distances and the same predecessors, which makes them easy to compare.

3.1 The version without a heap

The minimum is found by a linear search of the array `d[]`, among the reached, unfinished vertices. Each of the at most n steps makes one search in O(n), and every arc is relaxed once, when the vertex it leaves is extracted. The total time is O(n² + m), and the extra memory is O(n): the arrays `d`, `pred` and `done`.

sssp_en.cpp · dijkstraArray, lines 138–161

```cpp
// Dijkstra without a heap: at every step a linear scan picks the unfinished vertex with
// the smallest distance; then the arcs that leave it are relaxed.
Result dijkstraArray(const Graph& g, int s) {
    Result r(g.n);
    std::vector<char> done(g.n, 0);
    r.dist[s] = 0;
    for (int step = 0; step < g.n; step++) {
        int u = -1;
        for (int v = 0; v < g.n; v++)
            if (!done[v] && r.dist[v] < INF && (u < 0 || r.dist[v] < r.dist[u])) u = v;
        if (u < 0) break;                       // the other vertices are unreachable
        done[u] = 1;
        r.order.push_back(u);
        for (const Arc& a : g.adj[u]) {
            r.relax++;
            if (!done[a.to] && r.dist[u] + a.cost < r.dist[a.to]) {
                r.dist[a.to] = r.dist[u] + a.cost;
                r.pred[a.to] = u;
                r.updates++;
            }
        }
    }
    return r;
}
```

3.2 The version with a min-heap

The reached, unfinished vertices sit in a binary min-heap ordered by the pair (d[v], v). A vertex enters the heap when it is first reached (`push`), sifts up when its distance drops (`decrease`, also called *decrease-key*) and leaves when it is extracted (`pop`). To find a vertex in the heap in O(1), the class `MinHeap` keeps the position of every vertex in the array `pos[]`. The children of position i are 2i + 1 and 2i + 2, and its parent is (i − 1) / 2.

Each heap operation costs O(log n). There are at most n insertions, n extractions and m decreases, so the total time is O((n + m) log n). In Table 1, at step 3, vertices 4 and 6 get smaller distances while they are in the heap; at step 4, vertex 5 enters the heap and sifts up to the root, because d[5] = 19 < d[4] = 20.

sssp_en.cpp · class MinHeap, lines 84–134

```cpp
// Min-heap of vertices ordered by (d[v], v): on equal distances the vertex with the
// smaller index comes out first, as in the version without a heap. pos[v] is the
// position of v in the heap or -1, so that decrease finds the vertex in O(1). The
// children of position i are 2i + 1 and 2i + 2.
class MinHeap {
public:
    MinHeap(int n, const std::vector<Dist>& d) : d_(d), pos_(n, -1) { h_.reserve(n); }
    bool empty() const { return h_.empty(); }
    bool contains(int v) const { return pos_[v] >= 0; }
    void push(int v) {                // inserts v, with key d[v]
        h_.push_back(v);
        up(static_cast<int>(h_.size()) - 1);
    }
    void decrease(int v) { up(pos_[v]); }   // d[v] has just dropped: v sifts up
    int pop() {                       // extracts the vertex with the smallest distance
        int top = h_[0], last = h_.back();
        h_.pop_back();
        pos_[top] = -1;
        if (!h_.empty()) {
            h_[0] = last;
            down(0);
        }
        return top;
    }

private:
    const std::vector<Dist>& d_;
    std::vector<int> h_, pos_;
    bool less(int a, int b) const { return d_[a] < d_[b] || (d_[a] == d_[b] && a < b); }
    void place(int v, int i) { h_[i] = v; pos_[v] = i; }
    void up(int i) {
        int v = h_[i];
        while (i > 0 && less(v, h_[(i - 1) / 2])) {
            place(h_[(i - 1) / 2], i);
            i = (i - 1) / 2;
        }
        place(v, i);
    }
    void down(int i) {
        int v = h_[i], size = static_cast<int>(h_.size());
        for (;;) {
            int c = 2 * i + 1;
            if (c >= size) break;
            if (c + 1 < size && less(h_[c + 1], h_[c])) c++;
            if (!less(h_[c], v)) break;
            place(h_[c], i);
            i = c;
        }
        place(v, i);
    }
};
```

sssp_en.cpp · dijkstraHeap, lines 163–188

```cpp
// Dijkstra with a min-heap: the minimum is extracted from the heap in O(log n); a
// vertex enters the heap when it is first reached and sifts up when its distance
// drops.
Result dijkstraHeap(const Graph& g, int s) {
    Result r(g.n);
    std::vector<char> done(g.n, 0);
    MinHeap h(g.n, r.dist);
    r.dist[s] = 0;
    h.push(s);
    while (!h.empty()) {
        int u = h.pop();
        done[u] = 1;
        r.order.push_back(u);
        for (const Arc& a : g.adj[u]) {
            r.relax++;
            if (!done[a.to] && r.dist[u] + a.cost < r.dist[a.to]) {
                r.dist[a.to] = r.dist[u] + a.cost;
                r.pred[a.to] = u;
                r.updates++;
                if (h.contains(a.to)) h.decrease(a.to);
                else h.push(a.to);
            }
        }
    }
    return r;
}
```

An alternative without `pos[]` inserts the vertex again at every decrease and skips, on extraction, the stale pairs whose distance is larger than `d[v]`. This is how the `std::set` version in the course repository [6] works, and the `std::priority_queue` version of exercise 1. With a Fibonacci heap the time drops to O(m + n log n) [4].

3.3 Which version, and when

Table 2. The cost of the two versions

VERSION	TOTAL COST	SPARSE GRAPH, M ≈ K·N	DENSE GRAPH, M ≈ N²
without a heap	$O(n^2 + m)$	$O(n^2)$	$O(n^2)$
with a min-heap	$O((n + m) \log n)$	$O(n \log n)$	$O(n^2 \log n)$

On sparse graphs the heap version clearly wins. On dense graphs its worst case is slower by a factor of log n, but that case appears only when many relaxations succeed. On random graphs with random costs few relaxations succeed, and the heap stays competitive even on dense graphs.

The doubling experiment, recommended by Sedgewick and Wayne [3], measures the time T(n) for n, 2n, 4n, ... If the ratio T(2n) / T(n) approaches 2^b, the time grows roughly as n^b: 2 means linear growth, a little over 2 means n log n, and 4 means n^2. In Table 3 the version without a heap has ratios of 4 and then 6, above the theoretical value: once the arrays no longer fit in the processor's cache, every access costs more. The heap version has ratios close to 2.4, as for n log n.

Table 3. An example of the results of option 10 of the program, with the default values: random graphs with m = 5n arcs, costs in [1, 100], compiled with `g++ -O2` and run on an Intel Xeon processor at 2.8 GHz. The times depend on the computer; the ratios much less so.

n	m	NO HEAP	RATIO	WITH HEAP	RATIO	BELLMAN-FORD	RATIO	PASSES
1000	5000	1.38 ms	–	0.125 ms	–	0.0868 ms	–	9
2000	10 000	5.00 ms	×3.6	0.278 ms	×2.2	0.215 ms	×2.5	10
4000	20 000	21.2 ms	×4.2	0.754 ms	×2.7	0.646 ms	×3.0	12
8000	40 000	132 ms	×6.2	1.82 ms	×2.4	1.26 ms	×2.0	12
16 000	80 000	808 ms	×6.1	4.38 ms	×2.4	2.57 ms	×2.0	12

4 The graph file and the generator

The program and the platform use the same text format. The first line holds n and m, and each of the next m lines holds one arc, "u v c": the tail, the head and the cost. Vertices are numbered from 1 to n; if an arc uses vertex 0, the file is read with the numbering 0..n − 1, like the test files of the course repository [6]. Lines that start with # are comments, and the numbers can be separated by spaces, commas or semicolons. Any error is reported with its line number.

```
# the graph of Figure 1
7 10
1 2 7
1 3 9
…
7 1 4
```

The generator (option 2 of the program) builds a graph with n vertices and m arcs, without loops or parallel arcs, with costs in [a, b]. The first n − 1 arcs form a tree rooted at 1, so every vertex can be reached from 1; then the arc list is shuffled. The program and the platform use the same random number generator, xorshift32 [7], so the same seed gives the same graph. The file `graf_mare.txt` is the graph generated with n = 10,000, m = 50,000, costs in [1, 100] and seed 2026.

The model files of the lab are `graf_mic.txt` (Figure 1), `graf_mare.txt`, `graf_negativ.txt`, `graf_ciclu_negativ.txt` and `graf_lant_invers.txt`; the last three are used in lab 2. On the platform, the same graphs are loaded from the "Model graphs" list, and any graph can be saved or opened from a file.

5 Lab work

1. **Visualizer, without a heap.** Load the small graph and run Dijkstra without a heap step by step, from vertex 1. Compare every step with Table 1 and Figure 2. Then pick "Path to" 5 and check the path 1 → 3 → 6 → 5, of cost 19.

2. **Visualizer, with a heap.** Run the same problem with the min-heap. Follow the heap array under the graph and find the steps with a *decrease-key*. Then run from source 2 and explain why vertex 1 keeps d = ∞.
3. **Practice.** Solve at least five Dijkstra tasks with no mistakes and no hints.
4. **Quiz.** Answer at least ten questions from the "Next vertex", "Distances", "Heap" and "Complexity" categories.
5. **The program.** Build the program of section 9, load `graf_mic.txt` with option 1 and run options 5 and 6. Check the path to vertex 5 with option 8.
6. **The large graph.** Load `graf_mare.txt` and compare the three algorithms with option 9. Do the same on the platform, in the "Timing" tab, with the "Load the large graph" button. Generate the graph again with option 2 and the parameters of section 4, save it with option 3 and compare the result with `graf_mare.txt`.
7. **Doubling.** Run option 10 with the default values, then with degree 0, for dense graphs. Plot the times in `doubling.csv` on a log–log scale and estimate the exponent of each version. Compare with the experiment on the platform.
8. **Going further.** Solve the exercises of section 6.

6 Exercises

1. **The STL priority queue.** Write `dijkstraPQ` with a `std::priority_queue<std::pair<Dist, int>>` in increasing order: at every decrease, insert the pair (d[v], v) again, and on extraction skip the pairs whose distance is larger than `d[v]`. Count the insertions and compare the times with `dijkstraHeap` on `graf_mare.txt` and in the doubling experiment.
2. **The std::set version.** Write the version of the course repository [6], in which a decrease erases the old pair from a `std::set` and inserts a new one. Compare it with the other two, on sparse and on dense graphs.
3. **Stopping at the target.** Change the heap version so that it stops when it extracts a target vertex t. On `graf_mare.txt`, measure how many vertices are extracted on average for 100 random targets.
4. **Undirected graphs.** Add to the menu the loading of an undirected graph, in which every edge "u v c" becomes the arcs u → v and v → u. Explain why an undirected edge with a negative cost leaves the problem without a solution.
5. **The number of shortest paths.** Along with the distances, compute the number of shortest paths from s to every vertex, modulo $10^9 + 7$. Test it on a 30 × 30 grid graph with every cost equal to 1, where the result can be checked with binomial coefficients.
6. **A d-ary heap.** Generalize `MinHeap` to a heap in which every node has d children, at positions d·i + 1, ..., d·i + d. Measure the times for d = 2, 4 and 8, on sparse and dense graphs. Explain why a larger d speeds up *decrease-key* but slows down extraction.
7. **Bonus: the worst case for the heap.** Build the dense graph with the arcs i → j, for all i < j, of cost 2(j − i) − 1. Show that every relaxation succeeds and compare the two versions. Why is the measured difference smaller than the factor log n suggests?

7 Applied questions

1. Run the algorithm on the graph of Figure 1 with source 2. In what order are the vertices extracted, and what distances do you get?
2. Add the arc 5 → 4, of cost 0, to the graph of Figure 1. What changes in Table 1, and what is the new shortest path to 4?
3. About how many elementary operations does each version make on a complete graph with n = 1000 vertices? And on a graph with $n = 10^6$ vertices and $m = 5 \cdot 10^6$ arcs?
4. What is the largest number of *decrease-key* operations in a graph with n vertices and m arcs? Give an example of a graph that reaches it.
5. On `graf_mare.txt`, with n = 10,000 and m = 50,000, how many times faster is the heap version, according to your own measurements with option 9? Compare with the ratio $n^2 / ((n + m) \log_2 n)$.
6. Run Dijkstra on `graf_negativ.txt`. Which distances are wrong, and at which step does the algorithm go wrong?

8 Review questions

1. What is the relaxation of an arc, and why does it never increase a distance?
2. Why does a vertex extracted by Dijkstra have its final distance when the costs are non-negative? Where does the proof use this assumption?
3. How is the shortest path to a vertex rebuilt from the array `pred[]`? Why do the arcs pred[v] → v form a tree?
4. Why does the min-heap need the positions of the vertices, `pos[]`? What alternatives are there?
5. On what kind of graph is the version without a heap preferable, and why?
6. What does a ratio T(2n) / T(n) of about 4 mean? And one of about 2.2?
7. Why does the program use the type `long long` for distances, with INF = LLONG_MAX / 4?

9 Source code

The complete menu-driven program starts from the two implementations of Dijkstra in the ADS repository of the course [6] and rewrites them in C++. Compared with them:

- the `std::set` version fixed the source at 1, and an unreachable vertex was printed with distance 0, like the source. Its "infinity" was 50,010,000, so a path longer than that was reported as missing;
- in the version without a heap of the other implementation, the distances were initialized only for the first m indices, and the search for the minimum went through the vertices 0..n − 1, so vertex n was never extracted. On the chain 1 → 2 → 3 → 4 → 5 it reported d[4] = d[5] = 0, and the array `visited` had a fixed 100 elements;
- in the heap version, the arcs of the extracted vertices were added to a list that was never cleared, so every step relaxed all the old arcs again and the list could overflow its array of N elements. The predecessors of the source's neighbours were never set, so the paths through them were cut short;

- the file `graph.h`, with the graph structure, was missing from the repository, and `conio.h`, `getch` and `system("cls")` tied the program to Windows.

The new program has a single graph structure, with adjacency lists and the arc list, `long long` distances, an indexed min-heap with *decrease-key*, path reconstruction, a warning for negative costs, the generator, saving and loading of graphs with validation, timing with `std::chrono`, and the doubling experiment, whose results go to `doubling.csv`. It runs on any system.

Build: `g++ -std=c++11 -O2 -Wall -Wextra -o sssp sssp_en.cpp`. Run: `./sssp` or `./sssp graf_mic.txt`.

sssp_en.cpp

```cpp
// Single-source shortest paths (SSSP) - DSA lab
//
// For a directed graph with arc costs and a source vertex s, the program computes the
// shortest distance d[v] from s to every vertex v and the predecessor of v on a
// shortest path:
//  - Dijkstra without a heap: the minimum is found by a linear scan of d, O(n^2 + m);
//  - Dijkstra with a min-heap: the reached, unfinished vertices sit in a min-heap,
//    O((n + m) log n);
//  - Bellman-Ford: at most n - 1 passes over all the arcs, O(n m); accepts negative
//    costs and detects the negative-cost cycles reachable from s.
// Dijkstra gives correct distances only if no arc has a negative cost.
//
// A graph file: n and m on the first line, then m lines "u v c", the arc u -> v of cost
// c. Vertices are numbered from 1 to n; the numbering 0..n-1 is accepted as well. Lines
// that start with # are comments.
//
// Build: g++ -std=c++11 -O2 -Wall -Wextra -o sssp sssp_en.cpp
// Run:   ./sssp   or   ./sssp graf_mic.txt

#include <algorithm>
#include <cerrno>
#include <chrono>
#include <climits>
#include <cmath>
#include <cstdint>
#include <cstdlib>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <unordered_set>
#include <vector>

typedef long long Dist;
const Dist INF = LLONG_MAX / 4;   // there is no path from s (yet)
const Dist NEG_INF = -INF;        // vertex reachable from a negative cycle
const int MAX_COST = 1000000000;  // |c| <= 10^9, so that sums fit in long long

struct Arc { int to, cost; };          // arc in the adjacency list of a vertex
struct Edge { int from, to, cost; };   // arc in the list of all arcs

struct Graph {
    int n = 0;                         // vertices are 0..n-1 in memory
    int base = 1;                      // numbering in the file: from 1 or from 0
    std::string note;                  // comment written on save (by the generator)
    std::vector<Edge> edges;           // the arcs, in file order (Bellman-Ford)
    std::vector<std::vector<Arc>> adj; // the adjacency lists (Dijkstra)

    void reset(int vertices, int numbering) {
        n = vertices;
        base = numbering;
        note.clear();
        edges.clear();
        adj.assign(n, std::vector<Arc>());
    }
    void add(int u, int v, int c) {
        edges.push_back(Edge{u, v, c});
        adj[u].push_back(Arc{v, c});
    }
    int m() const { return static_cast<int>(edges.size()); }
    int label(int v) const { return v + base; }
    bool hasNegative() const {
        for (const Edge& e : edges)
            if (e.cost < 0) return true;
        return false;
    }
};

struct Result {
    std::vector<Dist> dist;   // d[v]; INF if v is unreachable, NEG_INF if it is -inf
    std::vector<int> pred;    // predecessor of v on the shortest path; -1 for s
    std::vector<int> order;   // Dijkstra: the order in which vertices become final
    std::vector<int> cycle;   // Bellman-Ford: a negative cycle, if there is one
```

```
    long long relax = 0;      // arcs examined
    long long updates = 0;    // successful relaxations (distances decreased)
    int passes = 0;           // Bellman-Ford: passes over the arc list
    bool negCycle = false;    // Bellman-Ford: a negative cycle is reachable from s
    explicit Result(int n) : dist(n, INF), pred(n, -1) {}
};

// ------------------------------------------------------------ min-heap

// Min-heap of vertices ordered by (d[v], v): on equal distances the vertex with the
// smaller index comes out first, as in the version without a heap. pos[v] is the
// position of v in the heap or -1, so that decrease finds the vertex in O(1). The
// children of position i are 2i + 1 and 2i + 2.
class MinHeap {
public:
    MinHeap(int n, const std::vector<Dist>& d) : d_(d), pos_(n, -1) { h_.reserve(n); }
    bool empty() const { return h_.empty(); }
    bool contains(int v) const { return pos_[v] >= 0; }
    void push(int v) {                // inserts v, with key d[v]
        h_.push_back(v);
        up(static_cast<int>(h_.size()) - 1);
    }
    void decrease(int v) { up(pos_[v]); }   // d[v] has just dropped: v sifts up
    int pop() {                       // extracts the vertex with the smallest distance
        int top = h_[0], last = h_.back();
        h_.pop_back();
        pos_[top] = -1;
        if (!h_.empty()) {
            h_[0] = last;
            down(0);
        }
        return top;
    }

private:
    const std::vector<Dist>& d_;
    std::vector<int> h_, pos_;
    bool less(int a, int b) const { return d_[a] < d_[b] || (d_[a] == d_[b] && a < b); }
    void place(int v, int i) { h_[i] = v; pos_[v] = i; }
    void up(int i) {
        int v = h_[i];
        while (i > 0 && less(v, h_[(i - 1) / 2])) {
            place(h_[(i - 1) / 2], i);
            i = (i - 1) / 2;
        }
        place(v, i);
    }
    void down(int i) {
        int v = h_[i], size = static_cast<int>(h_.size());
        for (;;) {
            int c = 2 * i + 1;
            if (c >= size) break;
            if (c + 1 < size && less(h_[c + 1], h_[c])) c++;
            if (!less(h_[c], v)) break;
            place(h_[c], i);
            i = c;
        }
        place(v, i);
    }
};

// ------------------------------------------------------------ algorithms

// Dijkstra without a heap: at every step a linear scan picks the unfinished vertex with
// the smallest distance; then the arcs that leave it are relaxed.
Result dijkstraArray(const Graph& g, int s) {
    Result r(g.n);
    std::vector<char> done(g.n, 0);
    r.dist[s] = 0;
    for (int step = 0; step < g.n; step++) {
        int u = -1;
        for (int v = 0; v < g.n; v++)
            if (!done[v] && r.dist[v] < INF && (u < 0 || r.dist[v] < r.dist[u])) u = v;
        if (u < 0) break;                       // the other vertices are unreachable
        done[u] = 1;
        r.order.push_back(u);
```

```
        for (const Arc& a : g.adj[u]) {
            r.relax++;
            if (!done[a.to] && r.dist[u] + a.cost < r.dist[a.to]) {
                r.dist[a.to] = r.dist[u] + a.cost;
                r.pred[a.to] = u;
                r.updates++;
            }
        }
    }
    return r;
}

// Dijkstra with a min-heap: the minimum is extracted from the heap in O(log n); a
// vertex enters the heap when it is first reached and sifts up when its distance
// drops.
Result dijkstraHeap(const Graph& g, int s) {
    Result r(g.n);
    std::vector<char> done(g.n, 0);
    MinHeap h(g.n, r.dist);
    r.dist[s] = 0;
    h.push(s);
    while (!h.empty()) {
        int u = h.pop();
        done[u] = 1;
        r.order.push_back(u);
        for (const Arc& a : g.adj[u]) {
            r.relax++;
            if (!done[a.to] && r.dist[u] + a.cost < r.dist[a.to]) {
                r.dist[a.to] = r.dist[u] + a.cost;
                r.pred[a.to] = u;
                r.updates++;
                if (h.contains(a.to)) h.decrease(a.to);
                else h.push(a.to);
            }
        }
    }
    return r;
}

// the vertices reachable from a vertex affected by a negative cycle have distance -inf
static void markNegInf(const Graph& g, std::vector<int> stack,
                       std::vector<Dist>& dist) {
    for (int v : stack) dist[v] = NEG_INF;
    while (!stack.empty()) {
        int u = stack.back();
        stack.pop_back();
        for (const Arc& a : g.adj[u])
            if (dist[a.to] != NEG_INF) {
                dist[a.to] = NEG_INF;
                stack.push_back(a.to);
            }
    }
}

// Bellman-Ford: every pass relaxes all the arcs, in the order of the list. After pass
// k, d[v] is at most the cost of any path from s to v with at most k arcs, so n - 1
// passes are enough. If a pass changes nothing, the distances are final.
Result bellmanFord(const Graph& g, int s) {
    Result r(g.n);
    r.dist[s] = 0;
    bool stable = false;
    for (int pass = 1; pass < g.n && !stable; pass++) {
        stable = true;
        for (const Edge& e : g.edges) {
            r.relax++;
            if (r.dist[e.from] != INF && r.dist[e.from] + e.cost < r.dist[e.to]) {
                r.dist[e.to] = r.dist[e.from] + e.cost;
                r.pred[e.to] = e.from;
                r.updates++;
                stable = false;
            }
        }
        r.passes = pass;
    }
    if (stable) return r;
    // pass n: an arc that can still be relaxed shows a negative cycle
```

```
    std::vector<int> bad;
    for (const Edge& e : g.edges) {
        r.relax++;
        if (r.dist[e.from] != INF && r.dist[e.from] + e.cost < r.dist[e.to]) {
            r.dist[e.to] = r.dist[e.from] + e.cost;
            r.pred[e.to] = e.from;
            bad.push_back(e.to);
        }
    }
    if (bad.empty()) return r;
    r.negCycle = true;
    int x = bad.back();                  // after n steps back, x lies on the cycle
    for (int i = 0; i < g.n && x >= 0; i++) x = r.pred[x];
    if (x >= 0) {
        int v = x;
        do {
            r.cycle.push_back(v);
            v = r.pred[v];
        } while (v != x && v >= 0 && r.cycle.size() <= static_cast<size_t>(g.n));
        std::reverse(r.cycle.begin(), r.cycle.end());   // in the direction of the arcs
        std::rotate(r.cycle.begin(),                    // start at the smallest vertex
                    std::min_element(r.cycle.begin(), r.cycle.end()), r.cycle.end());
        r.cycle.push_back(r.cycle[0]);
    }
    markNegInf(g, bad, r.dist);
    return r;
}

// the shortest path from s to t, rebuilt from the predecessors; empty if none is finite
static std::vector<int> pathTo(const Result& r, int t) {
    std::vector<int> p;
    if (r.dist[t] == INF || r.dist[t] == NEG_INF) return p;
    for (int v = t; v != -1; v = r.pred[v]) p.push_back(v);
    std::reverse(p.begin(), p.end());
    return p;
}

// ---------------------------------------------------------------- files

// "1 arc", "5 arcs"
static std::string plural(long long n, const char* one, const char* many) {
    return std::to_string(n) + " " + (n == 1 ? one : many);
}

static bool isSep(char c) {
    return c == ' ' || c == '\t' || c == ',' || c == ';' || c == '\r';
}

// reads the integers of a line; separators: spaces, commas, semicolons
static bool parseNums(const std::string& line, std::vector<long long>& out) {
    out.clear();
    size_t i = 0;
    while (i < line.size()) {
        if (isSep(line[i])) { i++; continue; }
        size_t j = i;
        while (j < line.size() && !isSep(line[j])) j++;
        std::string tok = line.substr(i, j - i);
        errno = 0;
        char* end = nullptr;
        long long v = std::strtoll(tok.c_str(), &end, 10);
        if (*end != '\0' || errno == ERANGE) return false;
        out.push_back(v);
        i = j;
    }
    return true;
}

static bool blankOrComment(const std::string& line) {
    for (char c : line) {
        if (c == '#') return true;
        if (!isSep(c)) return false;
    }
    return true;
}

// reads the graph from a file; on error returns false and explains in err
```

```
static bool loadGraph(const std::string& file, Graph& g, std::string& err) {
    std::ifstream in(file.c_str());
    if (!in) { err = "the file cannot be opened"; return false; }
    std::string line;
    std::vector<long long> v;
    long long n = -1, m = -1;
    std::vector<Edge> list;
    std::vector<int> lineOf;
    int no = 0, lo = INT_MAX;
    while (std::getline(in, line)) {
        no++;
        if (blankOrComment(line)) continue;
        std::string where = "line " + std::to_string(no) + ": ";
        if (!parseNums(line, v)) {
            err = where + "integers were expected";
            return false;
        }
        if (n < 0) {
            if (v.size() != 2) { err = where + "n and m were expected"; return false; }
            n = v[0];
            m = v[1];
            if (n < 1 || n > 1000000) {
                err = where + "n must be between 1 and 10^6";
                return false;
            }
            if (m < 0 || m > 50000000) {
                err = where + "m must be between 0 and 5*10^7";
                return false;
            }
            list.reserve(static_cast<size_t>(m));
            continue;
        }
        if (v.size() != 3) { err = where + "an arc was expected: u v c"; return false; }
        if (static_cast<long long>(list.size()) == m) {
            err = where + "the file has more than m = " + std::to_string(m) + " arcs";
            return false;
        }
        if (v[0] < INT_MIN || v[0] > INT_MAX || v[1] < INT_MIN || v[1] > INT_MAX) {
            err = where + "vertex out of range";
            return false;
        }
        if (v[2] < -MAX_COST || v[2] > MAX_COST) {
            err = where + "the cost must be between -10^9 and 10^9";
            return false;
        }
        Edge e = {static_cast<int>(v[0]), static_cast<int>(v[1]),
                  static_cast<int>(v[2])};
        lo = std::min(lo, std::min(e.from, e.to));
        list.push_back(e);
        lineOf.push_back(no);
    }
    if (n < 0) { err = "the file does not hold n and m"; return false; }
    if (static_cast<long long>(list.size()) != m) {
        long long k = static_cast<long long>(list.size());
        err = "the file has " + plural(k, "arc", "arcs") + ", not m = " +
              std::to_string(m);
        return false;
    }
    int base = (!list.empty() && lo == 0) ? 0 : 1;   // a vertex 0 means 0..n-1
    for (size_t i = 0; i < list.size(); i++) {
        const Edge& e = list[i];
        if (e.from < base || e.from >= base + n || e.to < base || e.to >= base + n) {
            err = "line " + std::to_string(lineOf[i]) + ": vertices must be between " +
                  std::to_string(base) + " and " + std::to_string(base + n - 1);
            return false;
        }
    }
    g.reset(static_cast<int>(n), base);
    for (const Edge& e : list) g.add(e.from - base, e.to - base, e.cost);
    return true;
}

static bool saveGraph(const std::string& file, const Graph& g) {
    std::ofstream out(file.c_str());
    if (!out) return false;
    if (!g.note.empty()) out << "# " << g.note << '\n';
```

```
    out << g.n << ' ' << g.m() << '\n';
    for (const Edge& e : g.edges)
        out << g.label(e.from) << ' ' << g.label(e.to) << ' ' << e.cost << '\n';
    return static_cast<bool>(out);
}

// ---------------------------------------------------------------- generator

// xorshift32 (Marsaglia, 2003): the platform uses the same generator, so the same seed
// gives the same graph in the program and on the platform
struct Rng {
    uint32_t x;
    explicit Rng(uint32_t seed) : x(seed ? seed : 2463534242u) {}
    uint32_t next() {
        x ^= x << 13;
        x ^= x >> 17;
        x ^= x << 5;
        return x;
    }
    int range(int a, int b) {            // an integer in [a, b]
        uint32_t span = static_cast<uint32_t>(static_cast<long long>(b) - a + 1);
        return static_cast<int>(a + static_cast<long long>(next() % span));
    }
};

// Random graph with n vertices and m arcs, without loops or parallel arcs, with every
// vertex reachable from vertex 1: the first n - 1 arcs form a tree rooted at 1, then
// the arc list is shuffled. Costs are in [a, b]. If a < 0 and noNegCycle is true, the
// cost of the arc u -> v is c + p[u] - p[v], with c in [0, b] and p[v] in [0, -a]: on a
// cycle the p terms cancel, so no cycle has a negative cost.
static bool generate(Graph& g, int n, long long m, int a, int b, uint32_t seed,
                     bool noNegCycle, std::string& err) {
    long long most = static_cast<long long>(n) * (n - 1) / 2;
    if (n < 1 || n > 1000000) {
        err = "n must be between 1 and 10^6";
        return false;
    }
    if (m < n - 1 || m > most || m > 5000000) {
        err = "m must be between n - 1 = " + std::to_string(n - 1) + " and " +
              std::to_string(std::min(most, 5000000LL));
        return false;
    }
    if (a > b || a < -1000000 || b > 1000000) {
        err = "the costs must satisfy -10^6 <= a <= b <= 10^6";
        return false;
    }
    bool shift = noNegCycle && a < 0;
    if (shift && b < 0) {
        err = "without negative cycles, b must be >= 0";
        return false;
    }
    Rng rng(seed);
    std::vector<int> perm(n - 1), p(n, 0);
    for (int i = 0; i < n - 1; i++) perm[i] = i + 1;
    for (int i = n - 2; i > 0; i--) std::swap(perm[i], perm[rng.range(0, i)]);
    if (shift)
        for (int v = 0; v < n; v++) p[v] = rng.range(0, -a);
    bool small = static_cast<unsigned long long>(n) * n <= 50000000ULL;
    std::vector<bool> mat(small ? static_cast<size_t>(n) * n : 0, false);
    std::unordered_set<unsigned long long> seen;
    auto key = [n](int u, int v) { return static_cast<unsigned long long>(u) * n + v; };
    auto taken = [&](int u, int v) {
        return small ? static_cast<bool>(mat[key(u, v)]) : seen.count(key(u, v)) > 0;
    };
    std::vector<Edge> list;
    list.reserve(static_cast<size_t>(m));
    auto take = [&](int u, int v) {
        if (small) mat[key(u, v)] = true; else seen.insert(key(u, v));
        int c = shift ? rng.range(0, b) + p[u] - p[v] : rng.range(a, b);
        list.push_back(Edge{u, v, c});
    };
    for (int i = 0; i < n - 1; i++) {    // the tree: perm[i] gets an earlier parent
        int r = rng.range(0, i);
        take(r == 0 ? 0 : perm[r - 1], perm[i]);
    }
    while (static_cast<long long>(list.size()) < m) {
```

```
        int u = rng.range(0, n - 1), v = rng.range(0, n - 1);
        if (u != v && !taken(u, v)) take(u, v);
    }
    for (int i = static_cast<int>(list.size()) - 1; i > 0; i--)
        std::swap(list[i], list[rng.range(0, i)]);
    g.reset(n, 1);
    for (const Edge& e : list) g.add(e.from, e.to, e.cost);
    std::ostringstream os;
    os << "random graph: n = " << n << ", m = " << m << ", costs in [" << a << ", " << b
       << "], seed " << seed << (shift ? ", no negative cycles" : "");
    g.note = os.str();
    return true;
}

// ---------------------------------------------------------------- output and timing

static std::string distStr(Dist d) {
    if (d == INF) return "inf";
    if (d == NEG_INF) return "-inf";
    return std::to_string(d);
}

// the time with 3 significant digits: "0.0123 ms", "1.23 ms", "123 ms"
static std::string timeStr(double ms) {
    std::ostringstream os;
    int digits = ms >= 100 ? 0 : ms >= 10 ? 1 : ms >= 1 ? 2 : ms >= 0.1 ? 3 : 4;
    os << std::fixed << std::setprecision(digits) << ms << " ms";
    return os.str();
}

static std::string pathStr(const Graph& g, const std::vector<int>& p) {
    std::string s;
    for (size_t i = 0; i < p.size(); i++)
        s += (i ? " -> " : "") + std::to_string(g.label(p[i]));
    return s;
}

static void printGraph(const Graph& g) {
    int neg = 0, cmin = INT_MAX, cmax = INT_MIN;
    for (const Edge& e : g.edges) {
        neg += e.cost < 0;
        cmin = std::min(cmin, e.cost);
        cmax = std::max(cmax, e.cost);
    }
    std::cout << "  " << plural(g.n, "vertex", "vertices") << ", "
              << plural(g.m(), "arc", "arcs");
    if (g.m() > 0) std::cout << ", costs in [" << cmin << ", " << cmax << "]";
    std::cout << ", " << plural(neg, "negative-cost arc", "negative-cost arcs") << '\n';
    int shown = g.m() <= 200 ? g.n : std::min(g.n, 20);
    for (int u = 0; u < shown; u++) {
        std::cout << "  " << std::setw(4) << g.label(u) << ":";
        if (g.adj[u].empty()) std::cout << " (no arcs)";
        for (const Arc& a : g.adj[u])
            std::cout << ' ' << g.label(a.to) << " (" << a.cost << ")";
        std::cout << '\n';
    }
    if (shown < g.n)
        std::cout << "  ... (" << plural(g.n - shown, "vertex", "vertices")
                  << " not shown)\n";
}

// the cost of the cycle; between two neighbours the cheapest arc is taken
static Dist cycleCost(const Graph& g, const std::vector<int>& c) {
    Dist sum = 0;
    for (size_t i = 0; i + 1 < c.size(); i++) {
        Dist best = INF;
        for (const Arc& a : g.adj[c[i]])
            if (a.to == c[i + 1]) best = std::min(best, static_cast<Dist>(a.cost));
        sum += best;
    }
    return sum;
}

static void printResult(const Graph& g, const Result& r, int s) {
    int reach = 0, far = -1;
    for (int v = 0; v < g.n; v++) {
```

```
        if (r.dist[v] == INF) continue;
        reach++;
        if (r.dist[v] != NEG_INF && (far < 0 || r.dist[v] > r.dist[far])) far = v;
    }
    std::cout << "  vertices reachable from " << g.label(s) << ": " << reach << " of "
              << g.n;
    if (far >= 0)
        std::cout << "; the farthest: " << g.label(far) << ", at distance "
                  << r.dist[far];
    std::cout << '\n';
    if (r.negCycle)
        std::cout << "  negative cycle: " << pathStr(g, r.cycle) << " (cost "
                  << cycleCost(g, r.cycle) << "); the vertices reachable from it have "
                  << "distance -inf\n";
    int shown = g.n <= 30 ? g.n : 10;
    std::cout << "  vertex   distance  predecessor  path\n";
    for (int v = 0; v < shown; v++) {
        bool none = r.pred[v] < 0 || r.dist[v] == NEG_INF;
        std::cout << "  " << std::setw(6) << g.label(v) << ' ' << std::setw(10)
                  << distStr(r.dist[v]) << ' ' << std::setw(12)
                  << (none ? std::string("-") : std::to_string(g.label(r.pred[v])))
                  << "  " << pathStr(g, pathTo(r, v)) << '\n';
    }
    if (shown < g.n)
        std::cout << "  ... (" << plural(g.n - shown, "vertex", "vertices")
                  << " not shown; option 8 prints the path to a vertex)\n";
}

typedef Result (*Algorithm)(const Graph&, int);
const char* const NAMES[3] = {"Dijkstra, no heap", "Dijkstra, min-heap",
                              "Bellman-Ford"};
const Algorithm ALGS[3] = {dijkstraArray, dijkstraHeap, bellmanFord};

// the mean time of one run, in milliseconds; runs repeat until 0.2 s have passed
static double timeMs(Algorithm alg, const Graph& g, int s, Result* keep) {
    typedef std::chrono::steady_clock Clock;
    int runs = 0;
    double total = 0;
    do {
        Clock::time_point t0 = Clock::now();
        Result r = alg(g, s);
        total += std::chrono::duration<double, std::milli>(Clock::now() - t0).count();
        if (runs++ == 0 && keep) *keep = r;
    } while (total < 200 && runs < 1000);
    return total / runs;
}

static int differences(const Result& a, const Result& b) {
    int k = 0;
    for (size_t v = 0; v < a.dist.size(); v++) k += a.dist[v] != b.dist[v];
    return k;
}

static std::string same(int diff) {
    if (diff == 0) return "the same distances";
    return plural(diff, "different distance", "different distances");
}

static void compareAll(const Graph& g, int s) {
    Result r[3] = {Result(0), Result(0), Result(0)};
    double t[3];
    for (int i = 0; i < 3; i++) t[i] = timeMs(ALGS[i], g, s, &r[i]);
    std::cout << "  algorithm                  time   arcs examined"
              << "  distances decreased\n";
    for (int i = 0; i < 3; i++) {
        std::cout << "  " << std::left << std::setw(21) << NAMES[i] << std::right
                  << std::setw(11) << timeStr(t[i]) << std::setw(16) << r[i].relax
                  << std::setw(20) << r[i].updates;
        if (i == 2) std::cout << "  (" << plural(r[i].passes, "pass", "passes")
                              << ")";
        std::cout << '\n';
    }
    std::cout << "  Dijkstra without and with a heap: "
              << same(differences(r[0], r[1])) << '\n';
    if (r[2].negCycle) {
        std::cout << "  Bellman-Ford found a negative cycle: Dijkstra's distances mean "
```

```
                    << "nothing.\n";
        return;
    }
    int d = differences(r[0], r[2]);
    std::cout << "  Dijkstra and Bellman-Ford: " << same(d);
    if (d) std::cout << " (Dijkstra is wrong because of the negative-cost arcs)";
    std::cout << '\n';
}

// The doubling experiment: random graphs with n0, 2 n0, 4 n0, ... vertices and costs in
// [1, 100]. If T(2n) / T(n) approaches 2^b, the time grows roughly as n^b.
static void doubling(int n0, int count, int degree) {
    std::ofstream csv("doubling.csv");
    csv << "n,m,dijkstra_array_ms,dijkstra_heap_ms,bellman_ford_ms,passes\n";
    std::cout << "        n           m   Dijkstra, no heap  Dijkstra, heap  "
              << "      Bellman-Ford\n";
    double prev[3] = {0, 0, 0};
    long long n = n0;
    for (int i = 0; i < count; i++, n *= 2) {
        long long m = std::max(n - 1, degree > 0 ? n * degree : n * (n - 1) / 4);
        Graph g;
        std::string err = "n too large";
        uint32_t seed = static_cast<uint32_t>(1 + i);
        bool ok = n <= 1000000;
        if (!ok || !generate(g, static_cast<int>(n), m, 1, 100, seed, false, err)) {
            std::cout << "  n = " << n << ": stopped (" << err << ")\n";
            break;
        }
        std::cout << std::setw(9) << n << std::setw(11) << m << std::flush;
        Result bf(0);
        double t[3];
        for (int k = 0; k < 3; k++) {
            t[k] = timeMs(ALGS[k], g, 0, k == 2 ? &bf : nullptr);
            std::ostringstream cell;
            cell << timeStr(t[k]);
            if (prev[k] > 0) cell << " x" << std::fixed << std::setprecision(1)
                                  << t[k] / prev[k];
            std::cout << std::setw(18) << cell.str() << std::flush;
            prev[k] = t[k];
        }
        std::cout << "  (" << plural(bf.passes, "pass", "passes") << ")\n";
        csv << n << ',' << m << ',' << t[0] << ',' << t[1] << ',' << t[2] << ','
            << bf.passes << '\n';
    }
    std::cout << "  xk = T(2n) / T(n). The results are also in doubling.csv.\n";
}

// ------------------------------------------------------------------ menu

enum Ask { OK, BAD, END };   // valid answer, invalid answer, end of input

static Ask askLine(const std::string& prompt, std::string& line) {
    std::cout << prompt;
    if (!std::getline(std::cin, line)) return END;
    size_t a = line.find_first_not_of(" \t\r"), b = line.find_last_not_of(" \t\r");
    line = a == std::string::npos ? "" : line.substr(a, b - a + 1);
    return OK;
}

// exactly one integer in [lo, hi]; *dflt is used for an empty line
static Ask askNum(const std::string& prompt, long long lo, long long hi, long long& x,
                  const long long* dflt = nullptr) {
    std::string line;
    if (askLine(prompt, line) == END) return END;
    if (line.empty() && dflt) { x = *dflt; return OK; }
    std::vector<long long> v;
    if (!parseNums(line, v) || v.size() != 1 || v[0] < lo || v[0] > hi) {
        std::cout << "Invalid value: enter one integer between " << lo << " and " << hi
                  << ".\n";
        return BAD;
    }
    x = v[0];
    return OK;
}

static Ask askYes(const std::string& prompt, bool& yes) {
```

```
    std::string line;
    if (askLine(prompt + " (y/n): ", line) == END) return END;
    if (line == "y" || line == "yes") { yes = true; return OK; }
    if (line == "n" || line == "no") { yes = false; return OK; }
    std::cout << "Answer y or n.\n";
    return BAD;
}

struct Session {
    Graph g;
    Result last = Result(0);   // the result of the last run (options 5-7)
    int lastAlg = -1, lastSrc = 0;
};

static Ask askVertex(const Graph& g, const std::string& prompt, int& v) {
    long long x = 0;
    Ask a = askNum(prompt, g.base, g.base + g.n - 1, x);
    v = static_cast<int>(x) - g.base;
    return a;
}

static Ask optLoadSave(Session& S, bool load) {
    std::string file, err;
    if (askLine("File name: ", file) == END) return END;
    if (file.empty()) { std::cout << "Empty name.\n"; return BAD; }
    if (!load) {
        std::cout << (saveGraph(file, S.g) ? "Saved the graph to " + file + ".\n"
                                           : "The file cannot be written.\n");
        return OK;
    }
    Graph h;
    if (!loadGraph(file, h, err)) {
        std::cout << file << ": " << err << ".\n";
        return BAD;
    }
    S.g = h;
    S.lastAlg = -1;
    std::cout << "Loaded the graph.\n";
    printGraph(S.g);
    return OK;
}

static Ask optGenerate(Session& S) {
    long long n = 0, m = 0, a = 0, b = 0, seed = 0, one = 1;
    bool avoid = false;
    Ask r = askNum("Number of vertices n: ", 1, 1000000, n);
    if (r == OK) r = askNum("Number of arcs m: ", 0, 5000000, m);
    if (r == OK) r = askNum("Minimum cost a: ", -1000000, 1000000, a);
    if (r == OK) r = askNum("Maximum cost b: ", -1000000, 1000000, b);
    if (r == OK) r = askNum("Seed (Enter = 1): ", 0, UINT32_MAX, seed, &one);
    if (r == OK && a < 0) r = askYes("No negative cycles?", avoid);
    if (r != OK) return r;
    Graph h;
    std::string err;
    if (!generate(h, static_cast<int>(n), m, static_cast<int>(a), static_cast<int>(b),
                  static_cast<uint32_t>(seed), avoid, err)) {
        std::cout << "Invalid parameters: " << err << ".\n";
        return BAD;
    }
    S.g = h;
    S.lastAlg = -1;
    std::cout << "Generated a " << S.g.note << ".\n";
    return OK;
}

static Ask optRun(Session& S, int k) {
    int s = 0;
    Ask r = askVertex(S.g, "Source vertex: ", s);
    if (r != OK) return r;
    if (k < 2 && S.g.hasNegative())
        std::cout << "Warning: the graph has negative-cost arcs, so Dijkstra may give "
                  << "wrong distances. Use Bellman-Ford.\n";
    double t = timeMs(ALGS[k], S.g, s, &S.last);
    S.lastAlg = k;
    S.lastSrc = s;
    std::cout << NAMES[k] << " from vertex " << S.g.label(s) << ": " << timeStr(t)
```

```
759                 << ", " << plural(S.last.relax, "arc examined", "arcs examined") << ", "
760                 << plural(S.last.updates, "distance decreased", "distances decreased");
761     if (k == 2) std::cout << ", " << plural(S.last.passes, "pass", "passes");
762     std::cout << ".\n";
763     printResult(S.g, S.last, s);
764     return OK;
765 }
766
767 static Ask optPath(Session& S) {
768     if (S.lastAlg < 0) {
769         std::cout << "Run one of the algorithms first (options 5-7).\n";
770         return BAD;
771     }
772     int t = 0;
773     Ask r = askVertex(S.g, "Target vertex: ", t);
774     if (r != OK) return r;
775     std::cout << "  " << NAMES[S.lastAlg] << ", from " << S.g.label(S.lastSrc) << " to "
776               << S.g.label(t) << ": ";
777     std::vector<int> p = pathTo(S.last, t);
778     if (S.last.dist[t] == INF)
779         std::cout << "there is no path.\n";
780     else if (S.last.dist[t] == NEG_INF)
781         std::cout << "distance -inf: a path can go around the negative cycle any "
782                   << "number of times.\n";
783     else
784         std::cout << "distance " << S.last.dist[t] << ", path " << pathStr(S.g, p)
785                   << " (" << plural(static_cast<long long>(p.size()) - 1, "arc", "arcs")
786                   << ").\n";
787     return OK;
788 }
789
790 static Ask optDoubling() {
791     long long n0 = 0, count = 0, degree = 0, d0 = 1000, c0 = 5, g0 = 5;
792     Ask r = askNum("Initial number of vertices n0 (Enter = 1000): ", 2, 1000000, n0,
793                    &d0);
794     if (r == OK) r = askNum("Number of graphs (Enter = 5): ", 1, 12, count, &c0);
795     if (r == OK)
796         r = askNum("Mean degree m / n, 0 for a dense graph (Enter = 5): ", 0, 1000,
797                    degree, &g0);
798     if (r != OK) return r;
799     doubling(static_cast<int>(n0), static_cast<int>(count), static_cast<int>(degree));
800     return OK;
801 }
802
803 int main(int argc, char** argv) {
804     Session S;
805     std::string err;
806     std::cout << "Single-source shortest paths - DSA lab\n";
807     if (argc > 1) {
808         if (loadGraph(argv[1], S.g, err))
809             std::cout << "Loaded " << argv[1] << ".\n";
810         else
811             std::cout << argv[1] << ": " << err << ".\n";
812     }
813     for (;;) {
814         std::cout << "\n=== graph: " << plural(S.g.n, "vertex", "vertices") << ", "
815                   << plural(S.g.m(), "arc", "arcs") << " ===\n"
816                   << " 1. Load a graph from a file\n"
817                   << " 2. Generate a random graph\n"
818                   << " 3. Save the graph to a file\n"
819                   << " 4. Print the graph\n"
820                   << " 5. Dijkstra without a heap, O(n^2 + m)\n"
821                   << " 6. Dijkstra with a min-heap, O((n + m) log n)\n"
822                   << " 7. Bellman-Ford, O(n m)\n"
823                   << " 8. Shortest path to a vertex, after options 5-7\n"
824                   << " 9. Compare the three algorithms on the current graph\n"
825                   << "10. The doubling experiment, on random graphs\n"
826                   << " 0. Exit\n";
827         long long o = 0;
828         Ask r = askNum("Option: ", 0, 10, o);
829         if (r == END) break;
830         if (r == BAD) continue;
831         if (o == 0) break;
832         if (o >= 3 && o <= 9 && S.g.n == 0) {
833             std::cout << "There is no graph yet: load one (1) or generate one (2).\n";
834             continue;
```

```
        }
        int s = 0;
        if (o == 1 || o == 3) {
            r = optLoadSave(S, o == 1);
        } else if (o == 2) {
            r = optGenerate(S);
        } else if (o == 4) {
            printGraph(S.g);
        } else if (o <= 7) {
            r = optRun(S, static_cast<int>(o - 5));
        } else if (o == 8) {
            r = optPath(S);
        } else if (o == 9) {
            r = askVertex(S.g, "Source vertex: ", s);
            if (r == OK) compareAll(S.g, s);
        } else {
            r = optDoubling();
        }
        if (r == END) break;
    }
    std::cout << "Goodbye!\n";
    return 0;
}
```

10 References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs", *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, chapter 24, "Single-Source Shortest Paths".

[3] R. Sedgewick, K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011, sections 1.4, "Analysis of Algorithms", and 4.4, "Shortest Paths".

[4] M. L. Fredman, R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms", *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.

[5] D. D. Burdescu, M. C. Mihăescu, *Algorithms and Data Structures*, Academica Greifswald.

[6] The ADS repository, `Graphs/Distances`: https://github.com/mihaescu/ADS/tree/master/Graphs/Distances.

[7] G. Marsaglia, "Xorshift RNGs", *Journal of Statistical Software*, vol. 8, no. 14, 2003.